XPoster.py Python Workflow, Function Architecture and Implementation Plan

COMPLETE PYTHON DESIGN SPECIFICATION

Revision purpose

This document turns the approved PowerShell-to-Python protocol and the reviewed Python workflow into a complete Python-side implementation specification. It defines the Python runtime sequence, function boundaries, function contracts, internal state, protocol-event ownership, error ownership, authentication boundary, media-upload loop, final post creation, debug routing, test plan, and implementation gate.

Authority and precedence

This document is subordinate to the approved application architecture and final user decisions. For Python-specific behavior, the following precedence applies:

1. Final user-approved decisions in the application design and protocol specification.
2. The explicit protocol update in this Python design, especially the DEBUG event behavior.
3. The Python workflow and function contracts in this document.
4. Earlier exploratory alternatives are historical context only.

Protocol update to the parent PDF

PROTOCOL UPDATE TO: "PS1-Python_JSONL_Protocol_and_Implementation_Plan_REVISED_COMPLETE-1.pdf"

Intentional Python debug/status messages are now structured JSON protocol events with Event=DEBUG and Source=PYTHON. PowerShell receives those events and writes them through the existing PowerShell logger. stderr is retained only for raw/unexpected Python runtime diagnostics that cannot safely be packaged as protocol JSON.

Important status labels

Label	Meaning
LOCKED	Already approved and should not be redesigned during coding.
IMPLEMENTATION DECISION	Chosen here to make the approved architecture executable.
VERIFY BEFORE CODING	Depends on the current X API, actual credential material, or existing local files that must be inspected before production code is written.

XPoster.py - Python Engine Design

Executive summary

The Python engine has one narrowly defined responsibility: receive one finalized post request from PowerShell, validate it, authenticate to X, upload zero to four media items sequentially, create one final post, and continuously report structured state back to PowerShell.

```
PowerShell -> Python stdin     : one complete JSON request
Python -> PowerShell stdout    : JSON-Lines protocol events
Python -> PowerShell stderr    : raw/unexpected runtime diagnostics only
Python process lifetime        : one post request per process
```

PowerShell remains the owner of the GUI, local history/export, repost behavior, process lifecycle, protocol parsing, user-visible status, and the final log file. Python owns the X API layer, request validation, credential use, authentication, sequential media upload, final post creation, and protocol-event emission.

Updated event vocabulary

```
READY
TEXT_COMPLETE
PROGRESS
MEDIA_COMPLETE
POSTING
DEBUG
SUCCESS
ERROR
```

DEBUG is nonterminal. It may appear between valid nonterminal events and does not advance the posting state machine. SUCCESS and ERROR remain terminal.

Python design principle

The Python implementation should be boring to read: one clear controller, small focused functions, no GUI logic, no Excel logic, no hidden retries, no silent repair of malformed requests, no secret values in output, and no ordinary print() calls to stdout.

1. Python workflow - successful run

```
START PYTHON
    |
    v
Receive ConfigPath from startup arguments
    |
    v
Read one complete JSON request from stdin
    |
    v
Extract RequestId as early as possible
    |
    v
Read and validate X_Config.txt
    |
    v
Validate complete request
    |
    v
Emit READY
    |
    v
Prepare / validate post text
    |
    v
Emit TEXT_COMPLETE
    |
    v
Load credentials
    |
    v
Authenticate / prepare X client
    |
    v
Process media sequentially (0..4)
    |
    v
Emit POSTING
    |
    v
Create final X post
    |
    v
Emit SUCCESS
    |
    v
Flush and exit 0
```

LOCKED: the PowerShell side has already saved/exported the finalized local record before Python is launched. Python does not own that local-save operation and must not delay or replace it.

Workflow invariant

- No X post is created until request validation, text preparation, authentication, and all required media uploads have succeeded.
- Media are uploaded sequentially in protocol v1.

- A failure at any nonterminal stage stops normal processing.
- Exactly one terminal application result is allowed: SUCCESS or ERROR.

2. Python workflow - failure path

```
ANY NONTERMINAL STAGE
    |
    v
Failure detected
    |
    v
Classify failure
    |
    +--> Emit DEBUG if a safe structured diagnostic is useful
    |
    +--> Write raw detail to stderr only when needed
    |
    v
Emit ERROR
    |
    v
Flush streams
    |
    v
Stop normal workflow
    |
    v
Exit non-zero
```

Python must never emit PROGRESS, MEDIA_COMPLETE, POSTING, DEBUG, or SUCCESS after ERROR. It must never emit ERROR after SUCCESS.

Failure ownership

Failure type	Owner	Result
Application/request/auth/upload/post failure	Python	Structured ERROR event, then non-zero exit.
Python executable cannot start	PowerShell	PowerShell synthetic PYTHON_START_FAILED.
Python crashes before structured ERROR	PowerShell	PowerShell synthetic PYTHON_EXITED_UNEXPECTEDLY.
Malformed/non-JSON stdout	PowerShell	Protocol failure; stop operation.
Wrong RequestId / illegal event order	PowerShell	Protocol failure; stop operation.

3. Protocol update - DEBUG event

This section is the explicit protocol update requested for the parent protocol PDF. Intentional Python debug/status output is no longer treated as ordinary stderr text. It is structured and sent over stdout as a valid JSON-Lines event.

DEBUG schema

Field	Required	Rule

Event	Yes	Always "DEBUG".
RequestId	Yes when known	Echo the active posting RequestId.
Source	Yes	Always "PYTHON" in protocol v1.
Level	Yes	Readable level such as INFO, WARNING, ERROR-DIAGNOSTIC.
Stage	Yes	Current Python stage.
Message	Yes	Safe human-readable diagnostic text. No secrets.
MediaId	Optional	Present when the debug message relates to one media item.
FileIndex	Optional	Present when useful for media correlation.

```
{"Event":"DEBUG","RequestId":"20261017_100055_001","Source":"PYTHON","Level":"INFO","
Stage":"MediaUpload","Message":"Starting upload","MediaId":1}
```

PowerShell should display the message in the Debug / Output area and write it through the existing PowerShell logger with Python identified as the source. DEBUG never changes the posting state by itself.

stderr after the DEBUG update

```
stdout = JSON protocol only, including DEBUG
stderr = raw/unexpected runtime diagnostics only
```

Examples suitable for stderr: an interpreter-level traceback, library noise that bypassed our event emitter, or a catastrophic serialization failure. Ordinary controlled progress and diagnostics should use DEBUG instead.

4. Event sequencing with DEBUG

DEBUG can be interleaved between valid nonterminal events without advancing the state machine.

```
READY
DEBUG Stage=TextPreparation
TEXT_COMPLETE
DEBUG Stage=Authentication
DEBUG Stage=MediaUpload MediaId=1
PROGRESS MediaId=1 Percent=...
PROGRESS MediaId=1 Percent=...
MEDIA_COMPLETE MediaId=1
DEBUG Stage=MediaUpload MediaId=2
PROGRESS MediaId=2 Percent=...
MEDIA_COMPLETE MediaId=2
POSTING
DEBUG Stage=PostCreate
SUCCESS
```

State validation therefore treats DEBUG as an informational event valid at any nonterminal stage, while preserving strict ordering for READY, TEXT_COMPLETE, media events, POSTING, and the terminal result.

Text-only sequence

```
READY
[DEBUG ...]
```

```
TEXT_COMPLETE
[DEBUG ...]
POSTING
[DEBUG ...]
SUCCESS
```

Media-only sequence

```
READY
TEXT_COMPLETE TextPresent=false
[DEBUG ...]
PROGRESS ...
MEDIA_COMPLETE
POSTING
SUCCESS
```

5. Internal Python architecture

```
Main
|
+-- ReadConfig
|   +-- ValidateConfig
|
+-- ReadRequest
|   +-- ValidateRequest
|       +-- ValidateMediaItem
|
+-- EmitEvent -> READY
|
+-- PrepareText
+-- EmitEvent -> TEXT_COMPLETE
|
+-- LoadCredentials
|   +-- ValidateCredentials
|
+-- AuthenticateX
|
+-- UploadMediaCollection
|   +-- UploadMedia
|       +-- EmitDebug
|       +-- EmitEvent -> PROGRESS
|       +-- EmitEvent -> MEDIA_COMPLETE
|
+-- EmitEvent -> POSTING
+-- CreatePost
+-- EmitEvent -> SUCCESS
+-- ExitApplication(0)

ANY FAILURE
+-- ClassifyError
+-- EmitDebug / stderr when useful
+-- EmitError
+-- ExitApplication(non-zero)
```

This is intentionally a small function set. Functions are separated by responsibility, not merely to create abstraction. The X-specific implementation is isolated behind AuthenticateX, UploadMedia, and CreatePost.

6. Function contract - Main

Item	Definition
Purpose	Own the complete Python workflow and call subordinate functions in the approved order.
Startup input	ConfigPath from the Python command-line/startup arguments.
Posting input	One complete JSON request read from stdin.
Normal result	Emit SUCCESS and exit 0.
Failure result	Classify failure, emit ERROR, exit non-zero.
Must not do	GUI work, Excel/history work, direct PowerShell logging, media parallelism, automatic repost retries.

Main is a controller, not a dumping ground. It should show the application sequence clearly while delegating parsing, validation, authentication, media upload, event formatting, and X posting to dedicated functions.

Main conceptual flow

```
config = ReadConfig(ConfigPath)
request = ReadRequest(stdin)
ValidateConfig(config)
ValidateRequest(request)
EmitEvent(READY)
prepared_text = PrepareText(request.Text)
EmitEvent(TEXT_COMPLETE)
credentials = LoadCredentials(config.KeysFilePath)
ValidateCredentials(credentials)
x_client = AuthenticateX(credentials, config)
x_media_ids = UploadMediaCollection(x_client, request.Media, request.RequestId)
EmitEvent(POSTING)
post_result = CreatePost(x_client, prepared_text, x_media_ids)
EmitEvent(SUCCESS)
ExitApplication(0)
```

The pseudocode is explanatory only; production code will follow the coding protocol later.

7. Function contracts - configuration

Function	Inputs	Returns	Responsibility
ReadConfig	ConfigPath	Config	Read key=value configuration without inventing alternate paths.
ValidateConfig	Config	valid / controlled error	Validate only Python-relevant required values and paths.

ReadConfig

- Reads the same X_Config.txt used by PowerShell.
- Ignores empty lines and comment lines if present.

- Preserves literal values exactly, including "Per run" even though Python does not use the logger switch directly.
- Does not silently substitute hard-coded paths.

ValidateConfig

Python should validate only what it actually requires. At minimum that includes KeysFilePath and any values necessary for the verified X implementation. PowerShell-only GUI/history paths do not need artificial validation inside Python unless the final implementation uses them.

```
LOCKED config path:
Z:\InsPro\Tools\Data\X_Post\Operational\X_Config.txt

Python receives this path from PowerShell at launch; it should not duplicate the path
internally.
```

8. Function contracts - request parsing and validation

Function	Inputs	Returns	Responsibility
ReadRequest	stdin	Request	Read exactly one JSON line and decode it.
ValidateRequest	Request	valid / controlled error	Validate protocol and complete request consistency.
ValidateMediaItem	MediaItem, expected index, total	valid / controlled error	Validate one media object and its local file.

ValidateRequest rules

- ProtocolVersion must be supported.
- RequestId must be present and non-empty.
- Text must exist as a field and may be empty.
- TotalFiles must be between 0 and 4.
- Media must exist as an array and its length must equal TotalFiles.
- At least one of Text or Media must contain content.
- MediaId values must be unique.
- FileIndex values must be sequential and consistent with TotalFiles.
- Every media item must pass ValidateMediaItem.

Python must not silently repair malformed input. If PowerShell sends an inconsistent request, the correct behavior is to produce a controlled protocol error so the defect is visible.

9. Function contract - ValidateMediaItem

Check	Status	Rule
MediaId	LOCKED	Valid local protocol identity; unique within the request.
FileIndex	LOCKED	Sequential upload/display position.
FileName	LOCKED	Present for display/correlation.
FilePath	LOCKED	Absolute/local source path available to Python.
File exists/readable	IMPLEMENTATION DECISION	Validate before any remote operation.

Supported media type	VERIFY BEFORE CODING	Must match current X endpoint rules.
Maximum size/duration	VERIFY BEFORE CODING	Must match current X endpoint/account rules.

The purpose of ValidateMediaItem is to fail locally and clearly when the file cannot possibly be uploaded. X-specific limits must not be guessed from memory; they are verified before production upload code is written.

10. Function contracts - credentials and authentication

Function	Inputs	Returns	Responsibility
LoadCredentials	KeysFilePath	Credentials	Read the local credential representation.
ValidateCredentials	Credentials	valid / controlled error	Check required fields for the chosen auth method.
AuthenticateX	Credentials, Config	XClient	Create/validate the authenticated X API client/session.

Credential rule

VERIFY BEFORE CODING: the exact X_Keys.txt schema must be derived from the actual credential material/screenshots in the Library and the authentication method required by the current X endpoints. This document deliberately does not invent field names or token-refresh behavior.

Authentication boundary

All OAuth/token details remain behind AuthenticateX. The rest of the Python program receives an XClient abstraction and should not care whether the verified implementation uses OAuth 1.0a, OAuth 2.0 user context, access-token refresh, or a library wrapper.

- No credential value may appear in DEBUG, ERROR Message, stderr, or the PowerShell log.
- No Authorization header may be logged.
- Authentication failures become controlled ERROR events.
- Automatic credential repair or account-setting changes are out of scope.

11. Function contract - PrepareText

Item	Rule
Input	PostText from the validated request.
Output	PreparedText.
Allowed behavior	Preserve Unicode and line breaks; ensure correct string handling; validate current API constraints.
Forbidden behavior	Do not rewrite, summarize, spell-correct, trim meaningful content, or alter the user post.
Protocol result	Main emits TEXT_COMPLETE after preparation succeeds.

For a media-only post, Text may be empty. TEXT_COMPLETE is still emitted with TextPresent=false so the protocol sequence remains deterministic.

12. Function contracts - media collection and single upload

Function	Inputs	Returns	Responsibility
UploadMediaCollection	XClient, Media[], RequestId	XMediaIds[]	Sequential controller for all selected media.
UploadMedia	XClient, MediaItem, TotalFiles, RequestId	XMediaId	Upload exactly one media item and emit its progress/completion events.

UploadMediaCollection

```
XMediaIds = []
for MediaItem in request order:
    XMediaId = UploadMedia(...)
    append XMediaId
return XMediaIds
```

If there are no media items, the function returns an empty collection immediately. If any upload fails, the function stops and propagates the failure; later files are not attempted.

UploadMedia

- Owns one media file only.
- May emit DEBUG before/during the operation when useful.
- Emits truthful PROGRESS only when the underlying upload mechanism exposes measurable progress.
- Obtains the remote XMediaId.
- Emits MEDIA_COMPLETE only after successful upload completion.
- Returns XMediaId as a string.
- Never creates the final post.

13. Truthful progress design

LOCKED SAFETY RULE: Python reports only progress it can actually measure. The implementation must never fabricate percentages merely to animate the GUI.

The final X upload mechanism may expose one of several progress models: byte counts during a request body stream, chunk completion milestones, asynchronous remote processing states, or no useful granular progress at all. The production implementation must adapt to what is actually available.

Observed capability	Protocol behavior
Reliable byte totals and bytes sent	Emit Percent and optionally BytesUploaded/BytesTotal.
Chunked upload with known chunk count	Emit progress based on verified completed chunks.
Only coarse API milestones	Emit meaningful milestone updates; do not invent byte percentages.
No measurable upload progress	Use status/debug updates; only emit PROGRESS if its fields remain truthful.

VERIFY BEFORE CODING: actual progress semantics must be proven with the selected current X media endpoint and Python HTTP client.

14. Partial media failure policy

```
Media 1
  -> upload success
  -> MEDIA_COMPLETE XMediaId=A

Media 2
  -> PROGRESS ...
  -> failure
  -> ERROR

Result:
  no POSTING
  no final post
  no SUCCESS
```

- The posting operation fails as a whole.
- Already-returned XMediaIds remain diagnostic state for that attempt only.
- Python does not automatically reuse partial remote media IDs on a new attempt.
- A new manual Post action starts a new RequestId and a fresh upload sequence in v1.

15. Function contract - CreatePost

Item	Definition
Inputs	XClient, PreparedText, XMediaIds[]
Output	PostResult containing PostId and PostUrl.
Precondition	All required media uploads have completed successfully.
Caller behavior	Main emits POSTING immediately before CreatePost.
Success behavior	Return PostId/PostUrl; Main emits SUCCESS.
Failure behavior	Raise/classify a controlled post-creation failure; no automatic resend.

Uncertain remote outcome

A network failure after the final creation request may leave the remote result uncertain. Python must distinguish "known failure" from "result unknown" when the selected API/client makes that distinction possible. The conservative recovery rule remains: no automatic resend that could create a duplicate post.

16. Protocol output functions

Function	Inputs	Output	Rule
EmitEvent	EventName, RequestId, event fields	One JSON line to stdout	The single normal stdout writer. Compact JSON, newline, immediate flush.
EmitDebug	RequestId, Level, Stage, Message, optional media fields	DEBUG via EmitEvent	Always Source=PYTHON. Safe message only.
EmitError	RequestId, ClassifiedError	ERROR via EmitEvent	Terminal application event. Normal workflow stops.
WriteRawDiagnostic	safe/raw diagnostic text	stderr line	Only for raw/unexpected runtime diagnostics. Never secrets.

IMPLEMENTATION DECISION: use one controlled stdout emitter rather than direct print() calls throughout the program. This protects JSON-Lines framing and makes it much harder for debug text to corrupt the protocol.

stdout invariant

```
Every physical stdout line must be a complete valid protocol JSON object.
No banners.
No print("Uploading...").
No traceback.
No library debug output.
```

17. Error classification and ownership

Error code / class	Typical stage	Python action
MALFORMED_REQUEST_JSON	RequestValidation	ERROR when RequestId can be recovered; otherwise best-effort ERROR with null/unknown RequestId.
UNSUPPORTED_PROTOCOL_VERSION	RequestValidation	ERROR; stop.
INVALID_REQUEST / EMPTY_POST	RequestValidation	ERROR; stop.
MEDIA_FILE_NOT_FOUND / UNREADABLE	RequestValidation	ERROR with MediaId/FileIndex when known.
CONFIG_ERROR	Startup/Config	ERROR; stop.
CREDENTIAL_ERROR / AUTH_FAILED	Credentials/Auth	ERROR; stop.
RATE_LIMITED / NETWORK_ERROR	MediaUpload/PostCreate	ERROR with safe details; no automatic repost.
MEDIA_UPLOAD_FAILED	MediaUpload	ERROR with media identity; stop whole attempt.
POST_CREATE_FAILED	PostCreate	ERROR; no automatic resend.
POST_RESULT_UNKNOWN	PostCreate	ERROR; manual reconciliation before trying again.
INTERNAL_ERROR	Internal	Safe generic ERROR; raw detail may go to stderr.

ClassifyError converts implementation/library exceptions into stable external protocol semantics. Raw exception text should not become the user-facing Message by default.

18. Runtime state owned by Python

State	Purpose	Lifetime
Config	Parsed Python-relevant configuration values.	Process
RequestId	Correlation identifier echoed in events.	Posting attempt
Request	Complete validated post request.	Posting attempt
PreparedText	Validated text passed to CreatePost.	Posting attempt
Credentials	Credential values loaded from KeysFilePath.	Process / posting attempt
XClient	Authenticated X API client/session.	Posting attempt
CurrentStage	Used for debug/error classification.	Posting attempt
CurrentMedia	Current sequential media item.	Per media
XMediaIds[]	Remote media IDs collected after successful uploads.	Posting attempt

PostResult	Final PostId/PostUrl after successful creation.	Terminal success

Python does not own PowerShell GUI state such as CurrentFileProgress, PostInProgress, SelectedMedia[], or PublishedPostUrl. It reports events; PowerShell decides how those events update the GUI/application state.

19. Responsibility boundary - Python versus PowerShell

PowerShell owns	Python owns
WinForms GUI	Request validation
Attach / Remove / Post-button state	Config values required by Python
Local ExportPostRecord before Python	Credential loading and validation
Excel/history/Repost workflow	X authentication/client creation
Process start/stop and redirected streams	Sequential media upload
JSON event parsing and state validation	Truthful progress event emission
Debug/Output window	Final X post creation
Existing DebugLog.ps1 file logging	DEBUG/ERROR protocol message generation
Synthetic process/protocol failures	Python/application failure classification

This boundary is intentionally strict. It prevents Python from becoming a second application controller and prevents PowerShell from containing X API implementation details.

20. Python process lifetime

```
PowerShell starts XPoster.py
    |
    v
Python receives ConfigPath argument
    |
    v
PowerShell writes exactly one compact JSON request line to stdin
    |
    v
PowerShell flushes and closes stdin
    |
    v
Python processes exactly one posting attempt
    |
    v
Python emits SUCCESS or ERROR
    |
    v
Python exits
```

Version 1 is not a daemon. There is no request queue, no command multiplexing, no persistent socket, and no second post request within the same Python process. A later long-lived worker would be a separately versioned design.

21. Security and secret-handling rules

- Never emit API keys, API secrets, client secrets, access tokens, refresh tokens, authorization headers, or complete credential-file contents.
- DEBUG Message must be safe for the PowerShell log.
- ERROR Message must be safe for the PowerShell log.
- stderr must also avoid secrets even though it is raw diagnostics.
- Do not log the complete request body by default because it contains the user post text.
- Prefer metadata summaries such as RequestId, TextLength, TotalFiles, MediaId, FileName, stage, and error code.
- Remote numeric identifiers such as PostId and XMediaId should be treated as strings.

The credential screenshots/library material are an input for design verification only. Secret values should not be copied into this PDF or repeated into chat output.

22. VERIFY BEFORE CODING - external/API facts

Item	Why open	Required evidence
X_Keys.txt exact schema	The path is locked, but the exact credential field structure has not been finalized in text.	Inspect the Library credential screenshots/material and map only the fields required by the verified auth flow.
X auth method	Protocol design cannot prove which user-context method the selected current endpoints accept.	Current X API documentation plus account/app capability.
Media upload endpoint/version	Upload behavior affects XMediaId handling and progress.	Current X API documentation and a small controlled prototype.
Post-create endpoint	Must match the active developer tier/capabilities.	Current X API documentation/account capability.
Token refresh behavior	Depends on the verified auth flow and actual credential material.	Current auth documentation + controlled token test.
Truthful progress semantics	Depends on HTTP client and upload endpoint behavior.	Prototype one real media upload and observe measurable progress/milestones.
Media constraints	Type/size/duration limits may change.	Current X media documentation.

23. Python implementation work plan

Phase	Action	Acceptance condition
0 - Source lock	Use the parent protocol PDF plus this Python design as the implementation baseline.	No locked names/flow are redesigned while coding.
1 - Verify inputs	Inspect X_Config, credential material, Python 3.13 runtime, current X auth/media/post API facts.	All VERIFY BEFORE CODING items required for the next phase are resolved.
2 - Protocol harness	Implement request reader, validation, EmitEvent/EmitDebug/EmitError using fake remote actions.	PS1 receives valid realtime JSONL including DEBUG.
3 - Failure harness	Exercise malformed request, invalid file, fake auth/upload/post failures, internal exceptions.	Exactly one terminal result; no stdout corruption.
4 - Auth adapter	Implement LoadCredentials/ValidateCredentials/AuthenticateX using verified auth method.	Controlled authentication test passes without secret leakage.
5 - Real media upload	Implement UploadMedia sequentially with truthful progress/milestones.	XMediaId returned and MEDIA_COMPLETE correct for one file.
6 - Media collection	Prove 0, 1, 2 and 4 media sequential flow.	Stable MediaId/FileIndex and correct XMediaIds order.
7 - Final post creation	Implement CreatePost and	PostId/PostUrl returned to PowerShell.

	POSTING/SUCCESS behavior.	
8 - Fault/uncertain outcome	Test network/auth/rate-limit/partial upload/uncertain create result.	No unsafe automatic repost.
9 - Final integration	Run through PS1 GUI after local export.	End-to-end post works; Debug/Output and log are readable.
10 - Three-pass acceptance	Review syntax, logic, protocol, error logging, secret handling and runtime behavior three times.	Ready for personal-use production.

24. Python acceptance test matrix

ID	Scenario	Expected
PY-01	Valid text-only request	READY -> TEXT_COMPLETE -> POSTING -> SUCCESS.
PY-02	Media-only request	TEXT_COMPLETE with TextPresent=false, sequential media, then POSTING/SUCCESS.
PY-03	One fake media	Truthful PROGRESS -> MEDIA_COMPLETE -> POSTING -> SUCCESS.
PY-04	Four fake media	MediaId/FileIndex 1->2->3->4, no overlap/parallelism.
PY-05	DEBUG during every major stage	All DEBUG lines are valid JSON, Source=PYTHON, PS1 can log them.
PY-06	stderr during valid run	Raw stderr does not corrupt stdout protocol.
PY-07	Empty post	Controlled EMPTY_POST/INVALID_REQUEST error.
PY-08	Malformed JSON	Controlled/best-effort error and non-zero exit.
PY-09	Missing media file	ERROR before any X operation.
PY-10	Media 2 fails after media 1 succeeds	No POSTING/SUCCESS; attempt stops.
PY-11	Authentication fails	ERROR at Auth stage; no media/post request.
PY-12	Rate limit/network error during upload	Structured ERROR; no automatic retry.
PY-13	Known final create failure	POST_CREATE_FAILED; no SUCCESS.
PY-14	Uncertain final create result	POST_RESULT_UNKNOWN; no automatic resend.
PY-15	Unicode + multiline text	Round-trips unchanged in one physical JSON request line.
PY-16	Secret-scan test	No token/key/Authorization value appears in stdout, stderr, or log-facing Message.
PY-17	Unexpected exception	Safe INTERNAL_ERROR + raw diagnostic to stderr when possible.
PY-18	Terminal event enforcement	No protocol line after SUCCESS or ERROR.

25. Exact end-to-end example - two media files

The request contains the complete post. The response stream contains incremental events. DEBUG messages can appear between state-changing events.

```
PS1 -> stdin
{"ProtocolVersion":1,"RequestId":"20261017_100055_001","Text":"Example post","TotalFiles":2,"Media":
[{"MediaId":1,"FileIndex":1,"FileName":"image1.jpg","FilePath":"Z:\\...\\image1.jpg"},
{"MediaId":2,"FileIndex":2,"FileName":"image2.jpg","FilePath":"Z:\\...\\image2.jpg"}]}


Python -> stdout
{"Event":"READY","RequestId":"20261017_100055_001","ProtocolVersion":1}
{"Event":"DEBUG","RequestId":"20261017_100055_001","Source":"PYTHON","Level":"INFO","Stage":"TextPreparation
","Message":"Preparing post text"}
```

```
{"Event":"TEXT_COMPLETE","RequestId":"20261017_100055_001","TextPresent":true,"CharacterCount":12}
{"Event":"DEBUG","RequestId":"20261017_100055_001","Source":"PYTHON","Level":"INFO","Stage":"MediaUpload","M
essage":"Starting upload","MediaId":1}
{"Event":"PROGRESS","RequestId":"20261017_100055_001","MediaId":1,"FileIndex":1,"TotalFiles":2,"Percent":37,
"FileName":"image1.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_100055_001","MediaId":1,"FileIndex":1,"TotalFiles":2,"FileNa
me":"image1.jpg","XMediaId":"111..."}
{"Event":"DEBUG","RequestId":"20261017_100055_001","Source":"PYTHON","Level":"INFO","Stage":"MediaUpload","M
essage":"Starting upload","MediaId":2}
{"Event":"PROGRESS","RequestId":"20261017_100055_001","MediaId":2,"FileIndex":2,"TotalFiles":2,"Percent":42,
"FileName":"image2.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_100055_001","MediaId":2,"FileIndex":2,"TotalFiles":2,"FileNa
me":"image2.jpg","XMediaId":"222..."}
{"Event":"POSTING","RequestId":"20261017_100055_001","MediaCount":2}
{"Event":"SUCCESS","RequestId":"20261017_100055_001","PostId":"333...","PostUrl":"https://x.com/..."}
```

26. Review pass 1 - workflow and protocol consistency

Simulated user upload: the complete Python design draft is treated as an independently uploaded document and reviewed from scratch for consistency with the approved PS1/Python protocol.

- Confirmed one process / one request / sequential media / one stdout JSONL stream.
- Confirmed the full request contains Text, TotalFiles and all Media entries; per-file events remain incremental.
- Confirmed MediaId remains local and XMediaId remains remote.
- Confirmed SUCCESS/ERROR are terminal.
- Confirmed DEBUG is nonterminal and cannot silently alter workflow state.
- Confirmed intentional Python diagnostics now use DEBUG and stderr is reserved for raw/unexpected diagnostics.
- Confirmed Python never assumes ownership of GUI/history/repost behavior.

Pass 1 result: no workflow contradiction remains. The only intentional protocol change is the documented DEBUG event update.

27. Review pass 2 - failure, security, and recovery

Simulated user upload: the Review-1 revision is reviewed again from a failure/recovery and secret-handling perspective.

- Partial media completion cannot accidentally proceed to POSTING after a later upload failure.
- The design distinguishes Python application errors from PowerShell process/protocol synthetic failures.
- No automatic repost retry is permitted after uncertain final-post outcomes.
- DEBUG and ERROR messages are explicitly required to be log-safe.
- Raw stderr is still available for interpreter/library failures but is not a normal logging path.
- Credential structure and current auth/media endpoint facts remain VERIFY BEFORE CODING instead of being guessed.
- Truthful progress is enforced as a design rule.

Pass 2 result: recovery behavior is conservative, deterministic, and appropriate for a personal-use poster where avoiding duplicate posts is more important than automatic retry.

28. Review pass 3 - implementation handoff

Simulated user upload: the Review-2 revision is reviewed a third time as a handoff to a developer who did not participate in the conversation.

- The developer can identify the exact Python workflow without reconstructing conversation history.
- Every major function has a defined responsibility, inputs, outputs, and ownership boundary.
- The DEBUG protocol update is explicit and cannot be confused with the older stderr-debug wording in the parent PDF.
- External unknowns are isolated in VERIFY BEFORE CODING items.
- The implementation work plan starts with a fake protocol harness before touching the real X API.
- The acceptance matrix covers success, failure, protocol, Unicode, stderr, DEBUG, partial upload, auth, uncertain outcome and secret leakage.
- The function set remains small enough for direct debugging and maintenance.

Pass 3 result: this document is suitable as the Python implementation specification once the listed current-X/API and credential facts are verified.

29. Final locked Python design

```
XPoster.py

Input:
  ConfigPath from process arguments
  one complete JSON request from stdin

Output:
  stdout = protocol JSON Lines only
  stderr = raw/unexpected diagnostics only

Events:
  READY
  TEXT_COMPLETE
  PROGRESS
  MEDIA_COMPLETE
  POSTING
  DEBUG
  SUCCESS
  ERROR

Execution:
  one process
  one post request
  sequential media
  one final post
  no automatic repost retry
```

Function set

```
Main
ReadConfig
ValidateConfig
ReadRequest
ValidateRequest
```

```
ValidateMediaItem
LoadCredentials
ValidateCredentials
AuthenticateX
PrepareText
UploadMediaCollection
UploadMedia
CreatePost
EmitEvent
EmitDebug
ClassifyError
EmitError
WriteRawDiagnostic
ExitApplication
```

IMPLEMENTATION DECISION: WriteRawDiagnostic is included as a tiny central stderr helper so raw diagnostics do not become scattered direct writes throughout the code. It does not replace DEBUG and should be used rarely.

30. Final implementation gate

Do not start production X API code until all applicable boxes are satisfied:

- [ ] Parent PS1/Python protocol PDF and this Python design are treated as the implementation baseline.
- [ ] DEBUG event update is added to the PowerShell event router/logging behavior.
- [ ] X_Config.txt is readable using the locked path/config contract.
- [ ] X_Keys.txt / Library credential material has been inspected and mapped without copying secrets into design documents.
- [ ] Current X authentication method is verified.
- [ ] Current media-upload endpoint and constraints are verified.
- [ ] Current final post-create endpoint is verified.
- [ ] Fake protocol harness passes including DEBUG and stderr coexistence.
- [ ] Request validation and media validation tests pass.
- [ ] Real progress semantics have been proven truthful.
- [ ] One-media real upload succeeds and returns XMediaId.
- [ ] Text-only and media-only posting paths pass.
- [ ] Partial-media failure does not reach POSTING.
- [ ] Uncertain final-post outcome does not auto-resend.
- [ ] Secret-scan test passes.
- [ ] Syntax, logic, protocol, and error logging are each reviewed three times before final release.

Only after this gate is satisfied should the fake remote operations be replaced by the complete real X API engine.